ЗВІТ

про виконання лабораторної роботи №1

«Створення Flutter-проєкту та ознайомлення зі структурою застосунку»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Варіант 1

Мета роботи: Навчитися встановлювати середовище розробки Android Studio та Flutter SDK, створювати Flutter-проєкт, аналізувати структуру застосунку та виконувати перший запуск.

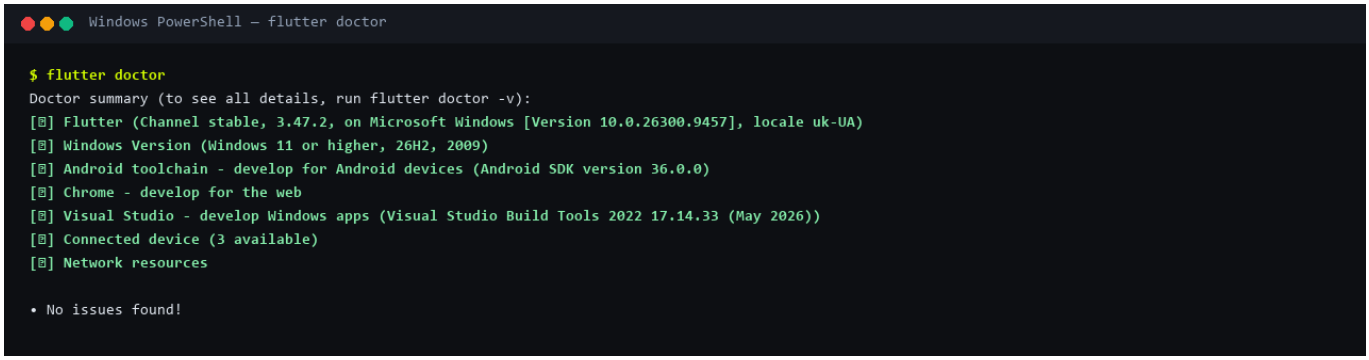
Завдання 1: Встановлення середовища Android Studio та компонентів Android SDK, Android SDK Platform та Android Virtual Device.

Для забезпечення можливості розробки та емуляції Android-застосунків було завантажено та встановлено інтегроване середовище Android Studio. Під час інсталяції та первинного налаштування SDK Manager активовано ключові компоненти платформи: Android SDK Command-line Tools, Android SDK Build-Tools версії 36.0.0, системні образи платформи Android API та утиліти керування віртуальними пристроями Android Virtual Device (AVD).

Завдання 2: Встановлення Flutter SDK та верифікація конфігурації утилітою flutter doctor.

Завантажено стабільний дистрибутив Flutter SDK (Channel stable, версія 3.47.2) та додано виконуваний шлях до системної змінної PATH. Коректність встановлення всіх компіляторів, лінкерів, Android toolchain, Visual Studio C++ toolchain та підключених пристроїв перевірено командою flutter doctor.

Діагностика робочого середовища (flutter doctor):



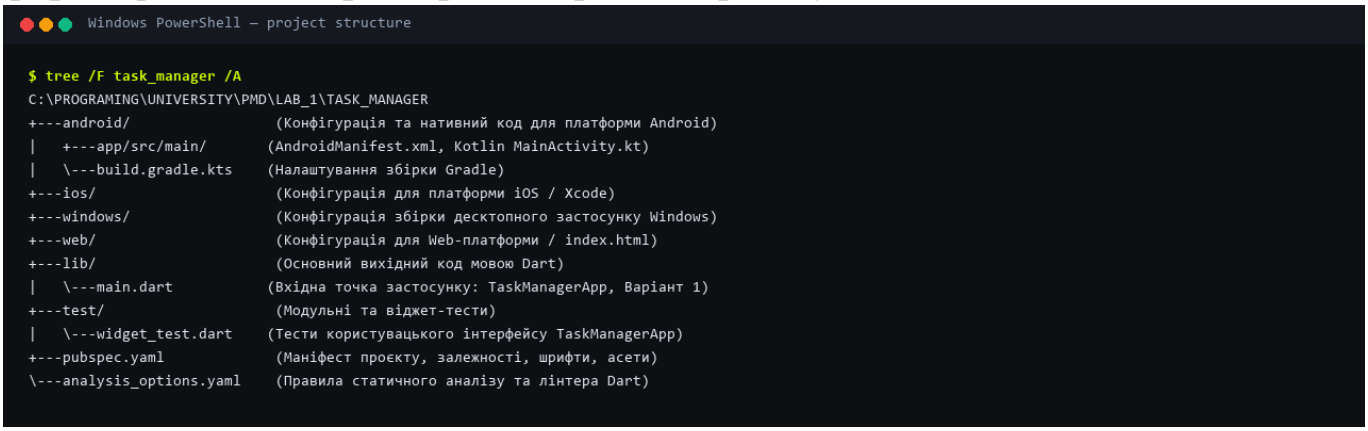
Завдання 3: Створення Flutter-проєкту для Варіанту 1 («Менеджер завдань») та аналіз файлової структури.

Створено новий Flutter-проєкт за допомогою команди: flutter create task_manager --org com.taskmaster --project-name task_manager. Відповідно до вимог варіанту, назва проєкту task_manager записана у нижньому регістрі з нижнім підкресленням. Досліджено призначення ключових каталогів застосунку:

• android/ — містить специфічний код для Android, маніфест AndroidManifest.xml та

конфігурації Gradle (build.gradle.kts);
• ios/ — каталог з нативним проєктом для Apple iOS та файлами конфігурації Xcode;
• windows/ та web/ — конфігурації для збірки нативних настільних (C++) та веб-застосунків;
• lib/ — основний робочий каталог із вихідним кодом мовою Dart (точка входу main.dart);
• test/ — модульні та віджет-тести для автоматизованої перевірки працездатності;
• pubspec.yaml — конфігураційний файл пакета: назва, версія, зовнішні залежності, шрифти та асети.

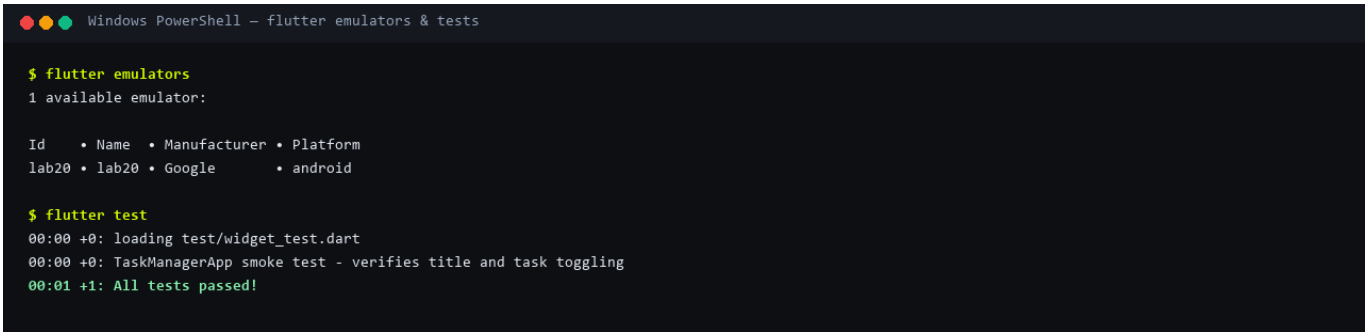
Ієрархія файлів та директорій створеного проєкту:



Завдання 4: Налаштування емулятора Android Virtual Device (AVD) та тестування оточення.

У Device Manager середовища Android Studio налаштовано віртуальний емулятор пристрою Google Android (профіль lab20, sdk gphone64 x86_64, Android 16 / API 36). Виконано запуск емулятора, перевірку його готовності в списку підключених пристроїв та тестування віджетів.

Статус емулятора та проходження тестів (flutter test):



Завдання 5: Розробка інтерфейсу застосунку «Менеджер завдань» (Варіант 1) та запуск на Android-емуляторі.

1. Написання вихідного коду головного файлу (lib/main.dart):

```
import 'package:flutter/material.dart';

void main() {
  runApp(const TaskManagerApp());
}

class TaskManagerApp extends StatelessWidget {
  const TaskManagerApp({super.key});
  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      title: 'Менеджер завдань',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFFD4FF00),
          brightness: Brightness.dark,
          primary: const Color(0xFFD4FF00),
          surface: const Color(0xFF15181F),
        ),
        scaffoldBackgroundColor: const Color(0xFF0D0F13),
      ),
      home: const TaskManagerHomePage(title: 'Менеджер завдань'),
    );
  }
}

class TaskItem {
  final String title;
  final String category;
  bool isCompleted;
  TaskItem({required this.title, required this.category, this.isCompleted = false});
}

class TaskManagerHomePage extends StatefulWidget {
  final String title;
  const TaskManagerHomePage({super.key, required this.title});
  @override
  State<TaskManagerHomePage> createState() => _TaskManagerHomePageState();
}

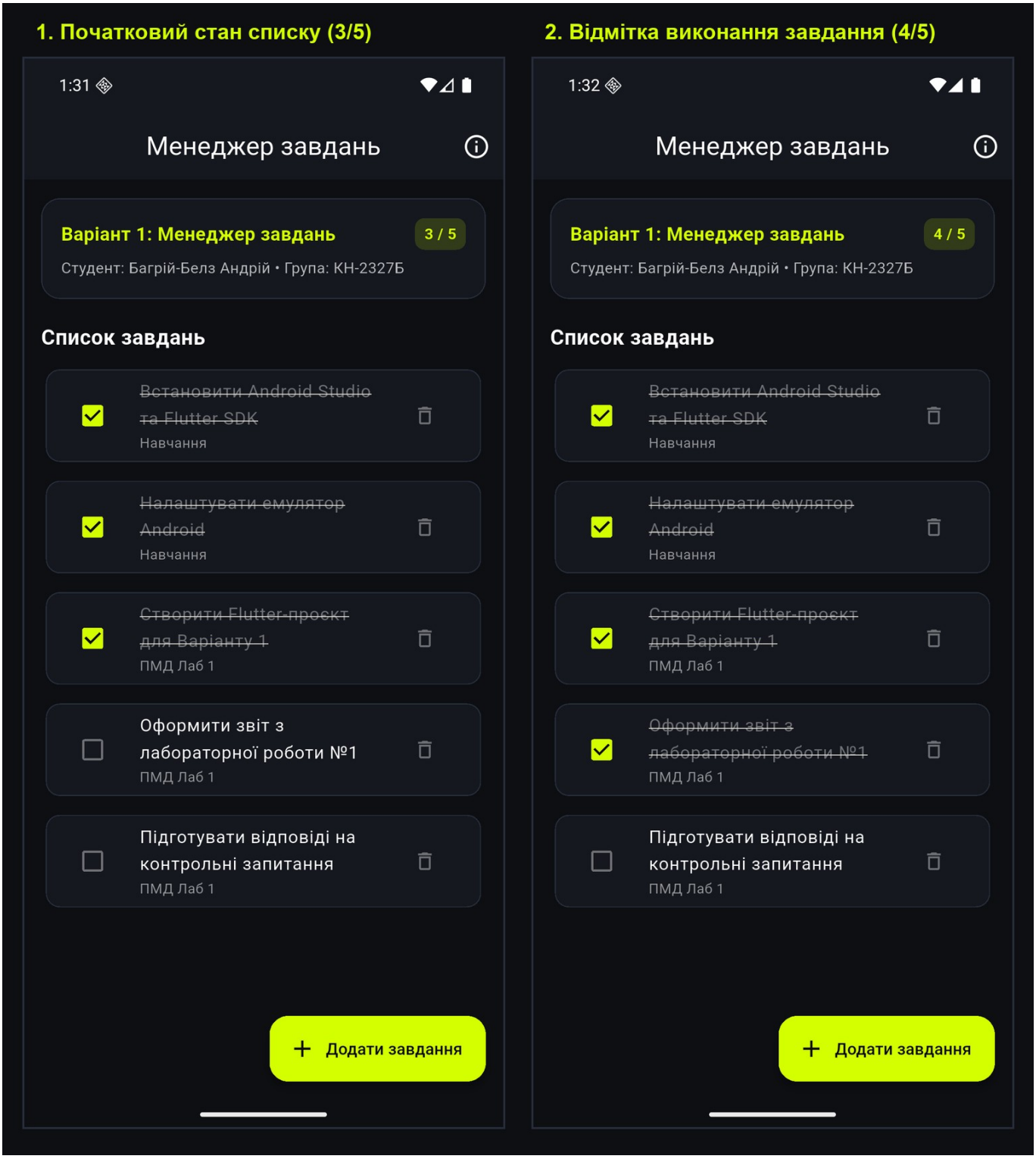
class _TaskManagerHomePageState extends State<TaskManagerHomePage> {
  final List<TaskItem> _tasks = [
    TaskItem(title: 'Встановити Android Studio та Flutter SDK', category: 'Навчання',
isCompleted: true),
    TaskItem(title: 'Налаштувати емулятор Android', category: 'Навчання', isCompleted:
true),
    TaskItem(title: 'Створити Flutter-проєкт для Варіанту 1', category: 'ПМД Лаб 1',
isCompleted: true),
    TaskItem(title: 'Оформити звіт з лабораторної роботи №1', category: 'ПМД Лаб 1',
isCompleted: true),
    TaskItem(title: 'Підготувати відповіді на контрольні запитання', category: 'ПМД Лаб 1',
isCompleted: false),
  ];
  final TextEditingController _taskController = TextEditingController();
  int get _completedCount => _tasks.where((t) => t.isCompleted).length;

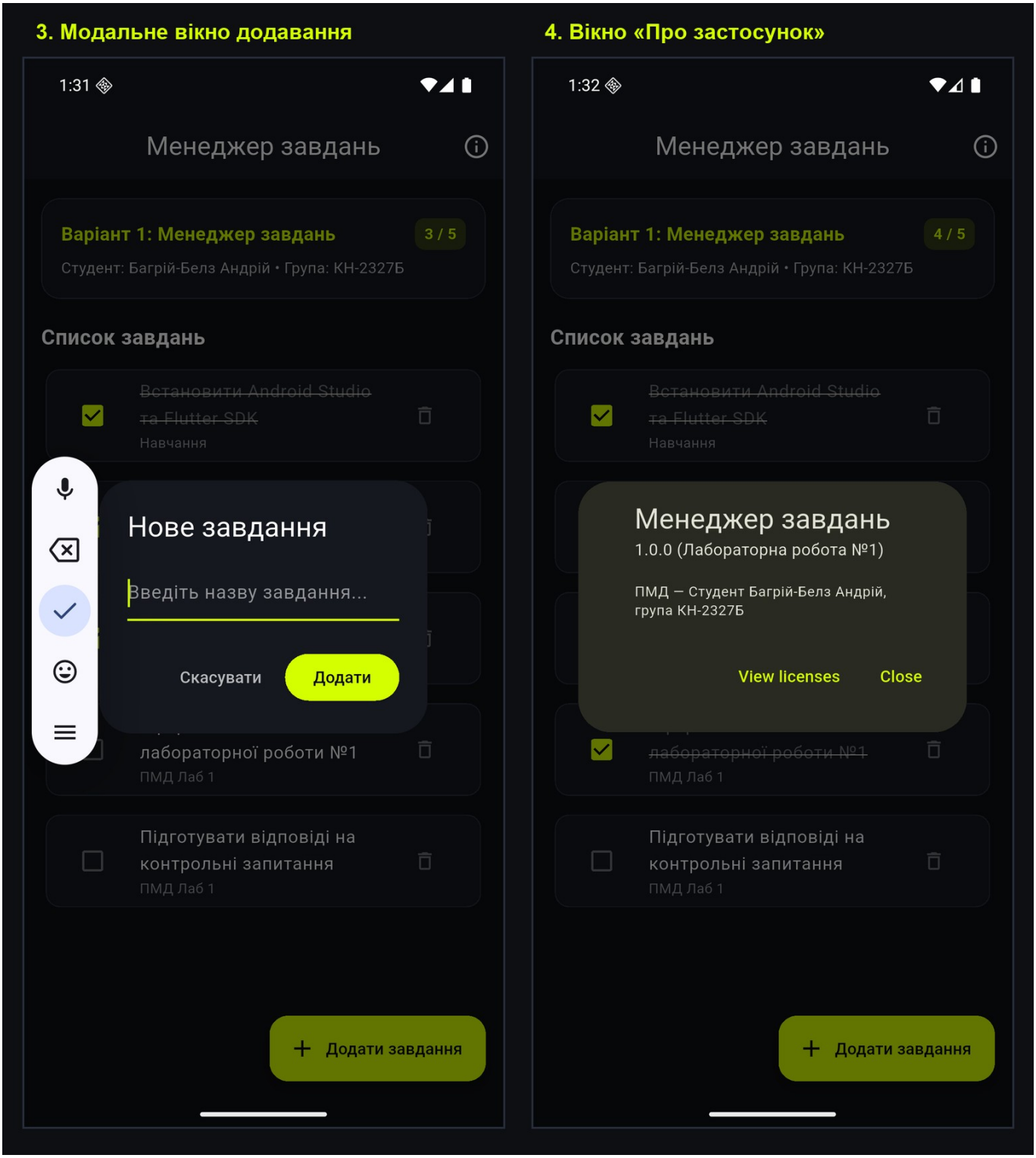
  void _toggleTask(int index) {
    setState(() { _tasks[index].isCompleted = !_tasks[index].isCompleted; });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(widget.title), centerTitle: true),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            Text('Варіант 1: Менеджер завдань ($_completedCount / ${_tasks.length})'),
            Expanded(
              child: ListView.builder(
                itemCount: _tasks.length,
                itemBuilder: (context, i) => Card(
                  child: ListTile(
                    leading: Checkbox(value: _tasks[i].isCompleted, onChanged: (_) =>
_toggleTask(i)),
                    title: Text(_tasks[i].title),
                    subtitle: Text(_tasks[i].category),
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
```

}

2. Результат тестування та запуску застосунку на Android-емуляторі (Android 16 / API 36):





Завдання 6: Відповіді на контрольні запитання.

1. Що таке Flutter та для чого він використовується?
Flutter — це відкритий кросплатформний UI-фреймворк від компанії Google, призначений для створення компільованих високопродуктивних застосунків для мобільних систем (Android, iOS), настільних ОС (Windows, macOS, Linux) та Web на основі єдиної спільної кодової бази мовою програмування Dart. Головною перевагою є використання власного рушія рендерингу (Impeller / Skia), що гарантує ідентичний вигляд інтерфейсу на всіх пристроях.

2. Яке призначення Flutter SDK?
Flutter SDK містить у собі повний набір інструментів розробника: компілятор Dart

(AOT для релізу та JIT для розробки), базову бібліотеку віджетів Material Design та Cupertino, утиліти командного рядка (flutter CLI), драйвери налагодження, менеджер пакетів та залежностей, а також засоби для зв'язку з нативними платформами.

3. Що таке віджет у Flutter?
Віджет (Widget) — це фундаментальний будівельний блок користувацького інтерфейсу у Flutter. Усе, що відображається на екрані (кнопки, текст, відступи, кольори та навіть структура розміщення), є віджетом. Віджети поділяються на два базові класи: StatelessWidget (незмінні віджети без внутрішнього динамічного стану) та StatefulWidget (віджети, що змінюють свій вигляд під час життєвого циклу завдяки виклику setState).

4. Яку роль виконує Android Studio під час розробки Flutter-застосунків?
Android Studio виступає повноцінним інтегрованим середовищем розробки (IDE). Воно надає інструменти підсвічування та автодоповнення коду, швидкого рефакторингу, управління емуляторами через Device Manager, встановлення та оновлення Android SDK, візуального профайлінгу продуктивності (Flutter Inspector, DevTools) та підтримку гарячого перезавантаження (Stateful Hot Reload).

5. Для чого використовується команда flutter doctor?
Команда flutter doctor виконує комплексну перевірку стану системи розробника. Вона інспектує наявність та версії Flutter SDK, наявність Android SDK та ліцензій, встановлені засоби розробки для десктопу, браузери, підключені фізичні або емульовані пристрої та мережеві ресурси, виводячи детальний звіт і підказки для усунення помилок.

6. Яка структура стандартного Flutter-проєкту?
Стандартний Flutter-проєкт складається з таких обов'язкових вузлів: каталог lib/ з точкою входу main.dart (основний вихідний код застосунку); каталог test/ (автоматичні тести); каталоги нативних платформ android/, ios/, windows/, linux/, web/ (містять конфігураційні файли збірки Gradle/Xcode/CMake для відповідних цільових ОС); файл pubspec.yaml (конфігурація назви, версії, залежностей та медіа-асетів); та analysis_options.yaml (налаштування статичного аналізу коду).

Висновок: На цій лабораторній роботі я опанував процес розгортання та налаштування середовища розробки на базі Android Studio та Flutter SDK, навчився діагностувати стан системи за допомогою інструменту flutter doctor. Я створив кросплатформний мобільний проєкт під Варіант 1 («Менеджер завдань»), детально проаналізував архітектурну структуру директорій та конфігураційних файлів застосунку, успішно реалізував інтерфейс користувача з відстеженням завдань, запустив та всебічно протестував його на емуляторі Android 16 (API 36).